

Cellula Week 6

Project 3.1 — Evaluator-Generator Workflow — Detailed Implementation Report

Team Members

Member	Name	Role
Member 1	Alaa AbdelHakeem	Data Ingestion & UI
Member 2	Zeina Mostafa	Core LLM Engineering
Member 3	Alaa Khamees	Orchestration, Caching & Feedback Loop

This report expands on the Project 3 team-division summary with a detailed account of what each member built. Member 3's section reflects the actual implemented code (reviewed and tested). Member 1 and Member 2's sections describe their assigned scope and responsibilities as defined in the project brief and workflow diagram; where their own code differs in specific implementation choices, their section should be treated as the source of truth.

Member 1 — Alaa AbdelHakeem

Data Ingestion & User Interface (The Frontend & Loader)

Responsible for the entry point of the platform: the Streamlit UI and the full ingestion pipeline that turns any supported source into retrievable knowledge.

1. User Interface

- Streamlit-based interface for uploading files and submitting URLs.
- Input controls for each supported source type: PDF, DOCX, TXT, source code, PPT/PPTX, web page URLs, Wikipedia pages, and WAV audio.
- Displays the user's question, the generated answer, and (per the workflow diagram) can surface iteration/loop status back to the user.

2. Supported Input Formats

- Documents: PDF, DOCX, TXT, PPT/PPTX — parsed and converted into plain text.
- Source code files — read and chunked in a way that preserves logical structure (functions/classes) where possible.
- Web pages via URL — fetched and stripped down to their main textual content.
- Wikipedia pages — fetched via a dedicated loader rather than generic URL scraping, to get cleaner article text.
- WAV audio — routed through a speech-to-text stage before entering the same text pipeline as every other source.

3. Ingestion Pipeline

- Load: read the raw file/URL/audio.
- Extract: pull out clean text from the source-specific format.
- Chunk: split extracted text into meaningfully sized, overlapping chunks suited for retrieval.
- Embed: generate vector embeddings for each chunk.
- Store: persist chunks + embeddings in a vector database so they can be retrieved by the Generator.

4. Speech-to-Text

WAV files are transcribed to text before entering the standard chunk/embed/store pipeline, so spoken content becomes searchable knowledge exactly like any document.

5. Interface with the rest of the system

Member 1's output is a retriever object (or equivalent function) capable of returning the top-k relevant chunks for a given question — this is the exact contract Member 3's orchestrator expects (see Retriever in interfaces.py, Member 3's section).

Note: exact library choices (e.g. which vector DB, which embedding model, which loaders) are Member 1's implementation decisions — refer to their code/comments for specifics.

Member 2 — Zeina Mostafa

Core LLM Engineering (Generator & Evaluator Prompts)

Responsible for designing the two LLM agents at the heart of the workflow, and their isolated memories.

1. Generator LLM

- Receives the user's question plus the context retrieved from the knowledge base (Member 1's layer).
- Formulates an answer grounded strictly in the retrieved context.
- Must explicitly state when the required information is not available in the provided sources, rather than hallucinating.
- On repeat iterations, also receives the Evaluator's feedback and produces a revised answer that addresses it.

2. Evaluator LLM

- Receives the user's original question together with the Generator's current answer.
- Judges the answer against: accuracy, relevance, completeness, consistency with the external knowledge, grounding in retrieved information, absence of unsupported claims, and overall quality.
- Produces a clear accept/needs-improvement decision.
- When rejecting an answer, produces concrete, actionable feedback describing what should change — this feedback is what gets passed back into the Generator on the next loop iteration.

3. Independent Memories

- Generator memory: stores conversation context, previous generated answers, retrieved context, and prior improvement attempts.
- Evaluator memory: stores evaluation history, previous feedback given, and past decisions.
- The two memories are managed as separate stores so the Evaluator's judgment isn't influenced by the Generator's internal state, and vice versa.

4. Interface with the rest of the system

Member 2's Generator and Evaluator are each expected to expose a simple call — generate(question, context, feedback, memory) and evaluate(question, answer, context, memory) — which is exactly what Member 3's orchestrator calls on every loop iteration (see GeneratorChain / EvaluatorChain in interfaces.py, Member 3's section).

Note: exact prompt wording, temperature settings, and evaluation scoring scheme are Member 2's implementation decisions — refer to their code/comments for specifics.

Member 3 — Alaa Khamees

Orchestration, Caching & Feedback Loop (The System Architecture)

Responsible for connecting the Generator and Evaluator into a single working system: the bounded feedback loop, the Redis caching layer, and the overall integration/logging. Delivered as a self-contained Python package at `src/backend/orchestrator/`, built against clear interfaces so it can be wired to Member 1's retriever and Member 2's LLM chains without further changes to its internal logic.

1. LCEL Orchestration (`orchestrator.py`)

- Each stage of a single pass — retrieve, generate, evaluate — is wrapped as a LangChain Runnable (RunnableLambda).
- The three stages are composed with LangChain Expression Language's `|` operator into one chain: `retrieve | generate | evaluate`.
- This chain is the literal unit of work executed on every loop iteration, satisfying the project's LCEL-based-orchestration requirement.
- The looping itself (since LCEL chains are static DAGs, not built for open-ended iteration) is handled in plain Python around the chain — this is the standard pattern for combining LCEL with an iterative agent loop.

2. The 4-Iteration Feedback Loop

- Implements: User Question → Generator → Answer → Evaluator → Decision.
- If accepted: loop exits immediately and the answer is returned.
- If rejected: the Evaluator's feedback is threaded into the next Generator call, and a loop counter increments.
- Hard cap of 4 iterations (configurable via `MAX_LOOP_ITERATIONS`), enforced in code — the loop cannot run indefinitely.
- If iteration 4 completes without acceptance, the system returns the latest answer with an explicit disclaimer that it could not be fully validated, rather than silently returning a possibly weak answer.
- Both the accepted path and the max-iteration-truncation path were unit tested against mock Generator/Evaluator chains to confirm correct loop counts and correct final-state handling.

3. Redis Caching Layer (`cache.py`)

- Caches four categories: embeddings, retrieval results, Generator (LLM) responses, and Evaluator results.
- Cache keys are content-addressed — a SHA-256 hash of the normalized inputs — so identical requests hit the cache regardless of timing, and any change to the underlying context or answer produces a different key (no stale hits).
- Generator-response keys incorporate the feedback text, so a revised answer after Evaluator feedback is never served the old cached version.
- Fails open: if Redis is unreachable, every cache method logs a warning and returns `None/no-ops` instead of crashing — caching is a performance optimization, not a correctness dependency. This was verified directly: the system ran correctly end-to-end with Redis unavailable.
- Includes a prefix-based invalidation method for when underlying documents are re-uploaded or knowledge changes.

4. Isolated Agent Memory (`memory.py`)

- GeneratorMemory and EvaluatorMemory are implemented as separate classes, each writing to its own namespaced Redis key (`mem:generator:{session}` vs `mem:evaluator:{session}`).
- Isolation is structural, not just a convention — there is no shared object reference between the two, so accidental cross-access is not possible by construction.
- Designed as a drop-in default: if Member 2's own memory classes expose the same method names (`add_turn` / `add_evaluation` / `get_history` / `clear`), they can replace these directly with no changes needed elsewhere.

5. Integration Contracts (interfaces.py)

- Defines Python Protocols for Retriever, GeneratorMemory, EvaluatorMemory, GeneratorChain, and EvaluatorChain — the exact seams between all three members' work.
- Lets the orchestration layer be built and tested before the other two members' real code is finished, since only these method signatures need to match, not any specific implementation.

6. Shared LLM Client (llm_client.py)

- Single get_llm() function wrapping LangChain's ChatOpenAI, configured for OpenRouter (OpenAI-compatible API).
- Centralizes model/provider configuration so switching models or providers is a one-line change in config.py, not a change scattered across every prompt file.

7. Configuration & Logging (config.py, logging_config.py)

- All settings (LLM provider/model, Redis connection, loop iteration cap, log level/location) are loaded from environment variables via a single config.py, with no secrets hardcoded.
- Centralized rotating-file + console logging, so every module's log calls land in one place with a consistent format for debugging and monitoring.

8. Testing & Validation Performed

- Verified the accepted-answer path: loop exits correctly the iteration an Evaluator approves, with the exact answer returned.
- Verified the max-iteration-truncation path: loop stops at exactly 4 iterations and appends the required disclaimer.
- Verified memory isolation: Generator memory contains only Generator-relevant fields, Evaluator memory only Evaluator-relevant fields, with no cross-contamination.
- Verified graceful degradation with Redis unavailable — no crashes, cache calls fail open as designed.

9. Deliverable Location

All code lives at src/backend/orchestrator/ in the repository: config.py, llm_client.py, interfaces.py, cache.py, memory.py, orchestrator.py, logging_config.py, stubs.py (placeholder retriever/Generator/Evaluator for standalone testing), and main.py (demo entrypoint). Root-level additions: .env.example and requirements additions (redis, python-dotenv).